Video transcript (about 16 minutes)

Read it as written. **[Brackets]** are actions, not speech. Slide numbers match the deck.

Before recording: - dashboard open and logged in: https://13-127-207-21.sslip.io/ui - Rocketlane, Slack and Gmail open in other tabs, phone nearby - the run emails ready to paste - the Mavis database details: host `postgres`, port `5432`, database `mavis_crm`, user `mavis_ro`, password from `mavis-db.txt`

---

1. Intro (30 s)

Hi, I'm Jai Krishna. For this assignment, I built an agentic, multi-agent system that runs NovaCRM's customer onboarding from end to end.

When a closed-deal email arrives, the system calls the account executive to confirm the plan, creates the Rocketlane project from the right template, opens a personalised Slack channel, and tracks the project until the customer is handed over to support.

Everything you will see is live on AWS and uses real Gmail, Retell, Rocketlane and Slack. I also went beyond the brief, because the agents verify the customer's data migration against the customer's own database.

2. The problem (30 s)

Today, this process is manual, and it breaks in four places.

Deal emails are messy. The plan tier is often unclear, so the wrong template gets used. The setup is done by hand, so steps get missed. And the data migration is marked as done without anyone checking it.

In my system, each of these problems is handled by an agent, and every agent works inside guardrails.

3. Architecture (1 min)

When a deal email arrives, Gmail pushes it to my FastAPI service, which starts a LangGraph run.

LangGraph coordinates the three agents and saves a checkpoint after every step in Postgres. This means a run can pause while it waits for the phone call or for a human decision, and then resume exactly where it stopped. It also means a crash never causes a step to run twice.

The agents never call an external service directly. Every action goes through a single tool registry, which talks to Retell, Rocketlane, Slack and Gmail. The React dashboard reads from the same database, and the whole system runs on AWS with Docker and HTTPS.

For the models, I use Claude Haiku for extraction, for reading the call transcript, and for drafting messages. On my evaluations, Haiku matched Sonnet's accuracy at about one seventh of the cost. I use Sonnet only for the handoff summary, because that task needs longer reasoning.

4. The agents (1 min)

The system has three agents. Each one acts on its own within its area, and each one has a hard boundary.

The Intake and Routing agent reads the email, calls the AE, and creates the project. It can never post to Slack, and it can never guess a tier.

The Communication agent owns every message that the customer sees. It cannot change the project or the tier.

The Project Tracking agent watches Rocketlane after the setup is done. It cannot write to the customer directly, so it asks the Communication agent to post for it.

Because of this split, the agent that reads untrusted email can never message a customer. The rule throughout the system is that the language model reads and drafts, the code makes every decision, and anything unclear goes to a human.

5. Flow 1: from email to project (1 min)

The intake flow has four checks, and every failed check leads somewhere safe.

First, the system checks that the sender is a known AE and that the email passes Gmail's DMARC check. If it does not, a human reviews it, and no AI call is ever made.

Second, it checks that all required fields are present and valid. If something is missing, the agent emails the AE with exactly what is missing, and nothing is created.

Third, it checks that the deal is new. The duplicate check runs before the call and again before the project is created.

Fourth, it checks that the tier is confirmed. Three signals must agree: Retell's call analysis, the AE saying yes when the plan is read back, and my own language model reading of the transcript. If they do not agree, the system calls again, up to three times, and then hands the run to a human. It never picks a tier by default.

When all four checks pass, an Enterprise deal gets the 30-day template with a dedicated CSM, and a Growth deal gets the 14-day template with a pooled CSM.

6. Flow 2: tracking (40 s)

The tracking agent hears about task changes through a Rocketlane webhook, and it also checks every project every 15 seconds.

When a phase starts or finishes, the customer receives an update in Slack. When the data import is done, the agent runs the migration checks. If someone ticks "Migration verified" without a passing check, the agent reopens the task. If a task is marked Blocked, the team is warned internally. When Go-Live is complete, the agent writes the handoff summary.

Overdue tasks are handled by Rocketlane's own automation. A task that is one day overdue notifies the project manager, and a task that is four days overdue notifies the project owner. This keeps working even if my service is down.

7. Tools, guardrails and idempotency (1 min 15 s)

Every action goes through the tool registry. The registry checks that the agent is allowed to use the tool, validates the input, reserves an idempotency key before anything happens, retries safely, and writes an audit record.

Idempotency matters because an API can time out after it has already done the work. Before retrying, my system checks whether the action already happened. As a result, each attempt makes exactly one call, and each run creates exactly one project, one channel and one welcome message. I proved this with a chaos test that injects up to 30 percent failures across 120 deals.

The guardrails are as follows. There is no project without a confirmed tier. The sender is verified before anything runs. Emails are treated as data and never as instructions, which protects against prompt injection. Duplicates are blocked. Every customer message is fact-checked, so nothing internal leaks. Everything is audited, and phone numbers are masked.

The system also protects itself. Every webhook is authenticated: Retell requests are signed, and Gmail and Rocketlane send a secret token. If a Retell webhook is lost, the scheduler fetches the call result from Retell directly. If a run gets stuck halfway through a step, it is escalated instead of hanging silently. A human's options are limited by the type of problem, so nobody can set a tier on a project that already exists.

The system also has more than 900 automated tests, and I will show the model evaluations after the demo.

8. Data migration (40 s)

This part goes beyond the brief. The CSM connects the customer's database in the dashboard by using a read-only user. The system tests the connection first, then encrypts the password and never shows it again.

The tracking agent then runs five checks against the data that landed in NovaCRM. It checks the row count, confirms that every record was copied, looks for duplicates, confirms that the required fields are filled, and makes sure no column was dropped. If the checks pass, the evidence is attached to the task. If they fail, the CSM is alerted, and a task that was ticked too early is reopened.

9. Live demo (about 6 min; say only these lines)

Positive run.

[Agents page.] All three agents are running and healthy.

[Send the Mavis Health email.] The email arrived through the Gmail push, and the sender and fields have passed their checks.

[Answer the call: "Enterprise", then "Yes".] The page shows the call while it is happening, and then it shows that the answer is being read.

[**AE calls** tab, play the recording.] Here is the recording, and the tier was confirmed by all three signals.

[Rocketlane, then open Members.] The project was created from the Enterprise template, which runs for 30 days with a dedicated CSM. The roles were filled in by code, based on the tier. I am the project owner, the project manager is assigned, and the dedicated CSM is assigned, while a Growth project would get the pooled CSM instead.

[Slack.] The channel and the welcome message are personalised, and they contain no price and nothing internal.

[Tick the Kickoff tasks in Rocketlane.] About 15 seconds later, the update appears in Slack, and the progress tab updates by itself.

[**Migration check** tab: connect Mavis's database.] All five checks fail, because rows are missing, one record is duplicated, emails are blank, and a column was dropped.

[**Fix destination data**, then **Run check**.] After the data is fixed, all five checks pass.

[Rocketlane, Automations.] This is the native overdue automation in Rocketlane.

Negative runs.

[Send the Quarry Labs email.] The contact email is missing, so the agent asked the AE for it, and nothing was created.

[Re-send the Mavis email.] This is a duplicate deal, so it went straight to a human, and no call was made.

[Send the Northwind Freight email, and say "wrong number" on the calls.] An unclear answer is never accepted. After three calls, the run went to a human with every recording attached. When a person sets the tier, that action is audited.

[Acme Retail.] Here, someone ticked "Migration verified" without a passing check, so the agent reopened the task.

[Terminal: `make test-part4`.] These are the Part 4 tests, and they all pass. They cover the happy path, validation, both templates, Rocketlane being down, an existing project, an AE who does not answer, and an ambiguous answer.

[AI usage, then Tool calls.] Every model call and every tool call is logged with its tokens, cost, inputs and outputs.

10. Evaluations (1 min)

I measured every prompt against real Claude by using labelled datasets. The evaluation suite exits with an error if any score drops below its threshold, so a prompt change cannot ship if it makes the results worse.

For extraction, both Haiku and Sonnet scored 34 out of 34. This dataset includes malformed emails, missing fields, several deals in one email, and prompt-injection attempts.

For call transcripts, both models scored 31 out of 31, and neither model ever confirmed a tier falsely. This dataset includes hedged answers, wrong numbers, voicemail, and trap calls where the AE only agreed to a plan that the voice agent had suggested.

For welcome messages, both models scored 8 out of 8. A separate Sonnet judge scored each message, and a code check also confirmed it.

Since Haiku matched Sonnet on all three tasks at about one seventh of the cost, Haiku runs the agents. The handoff summary is the one place where Sonnet did better, scoring 6 out of 6 compared with Haiku's 5 out of 6, so Sonnet writes only the handoff.

11. What testing taught me (1 min)

I logged every problem I found in an issues document, and it now has more than a hundred entries, each with its fix. Four of them show how I worked.

First, during a review of the deployed system, I realised that anyone could forge an email from an AE's address and trigger a real phone call. I fixed this by trusting only Gmail's own DMARC verdict.

Second, Rocketlane can time out after it has already created a project. A simple retry would have created a duplicate, so every write now checks whether it already happened before trying again.

Third, the stress tests showed that two customers with long, similar names could end up sharing one Slack channel. Each channel name now includes a short code that is unique to its run.

Fourth, on a live call, the speech recogniser heard my "Growth" as "group". The guardrail held, because the system never guessed and kept retrying. I then tuned the voice agent with Indian-English recognition, boosted the plan names, and added a yes-or-no confirmation. Afterwards, an independent review found that this new confirmation step could let a plan that only the bot had said slip through. I closed that gap, added trap cases to the evaluations, and confirmed that there were zero false confirmations.

12. Assumptions and design (1 min)

I made a few assumptions, and each one has a reason. Each email announces one deal, because that is how Sales works today. The tier comes from the phone call and never from the price, because prices in emails are unreliable. CS owns the templates, so they live in configuration and not in prompts. Overdue escalation stays in Rocketlane, so it works even if my service is down. Finally, customer data is only ever read and never written.

The design enforces five rules. Every side effect happens exactly once. A crash never loses or repeats work. Each agent can use only its own tools. A human decides anything that is unclear. And no prompt change ships without passing the evaluations. Every decision, guardrail and issue is documented in the repository.

Slack is real in this demo and is not mocked. The only gap in production is Slack Connect, which needs a paid plan, so that invite is flagged as a manual step.

To sum up, the system delivers everything the brief asked for. On top of that, it adds migration verification, live tracking, call recordings, observability, evaluations and chaos testing. Thank you for watching.